# Ders 11 Çalışma Özeti

## Ders 11 Çalışma Özeti

### Genel Konular

- Ağaç (tree) veri yapısı: neden gerekli, nerede kullanılır.
- Ağaç terminolojisi: kök, düğüm, kenar, iç düğüm, yaprak, derece, seviye, yükseklik, derinlik, alt ağaç, orman.
- Genel ağacın bilgisayarda saklanması: dizi ile ve bağlı liste (left child - right sibling) ile.
- İkili ağaç (binary tree) ve saklama yöntemleri: dizi ($2i + 1$, $2i + 2$) ve bağlı liste.
- Tam ikili ağaç (complete binary tree) ve dengeli ikili ağaç (balanced binary tree).
- Yüksekliği $h$ olan ikili ağaçta en çok/en az yaprak ve düğüm sayısı.
- Ağaç gezinme (traversal): preorder, postorder, inorder.
- İkili arama ağacı (binary search tree, BST): mantığı, karmaşıklığı, ekleme (insert) ve arama (search).
- Soru-cevap: çift pointer (pointer to pointer) nedir.

### Hocanın Özellikle Vurguladığı Kısımlar

- Dizi, bağlı liste, yığın, kuyruk doğrusal (linear) veri yapılarıdır. Ağaç doğrusal olmayan (non-linear) bir veri yapısıdır.
- Ağaç, bilginin hiyerarşik olarak saklanması ve erişilmesi gerektiğinde kullanılır.
  - Örnekler: dosya sistemi, aile ağacı, karar ağaçları (decision tree).
- Çocuk sayısı belli olmayan bir ağacı saklamanın en uygun yolu bağlı listedir.
- Dizi ile saklamada iki sorun var:
  - Bir düğümün en fazla kaç çocuğu olacağını önceden bilmek gerekir.
  - Boş kalan gözler yüzünden çok yer israf edilir.
- İkili ağaçta seçim uygulamaya göre yapılır.
  - Ağaçta az boşluk varsa (ör. tam ikili ağaç) dizi uygundur.
  - Çok boşluk varsa veya sık ekleme/silme yapılıyorsa bağlı liste uygundur.
- BST'de arama, ağaç dengeli ise $O(\log n)$ sürer. Ağaç tek tarafa yığılırsa en kötü durumda $O(n)$ olur.
- Bu yüzden sık ekleme/silme yapılan BST'lerde belli aralıklarla denge kontrol edilip yeniden sağlanmalıdır.
- BST'de ekleme her zaman yaprağa yapılır. Araya eklemek ek iş gerektirir.
- Hoca derse canlı katılımın ve soru sormanın önemini birkaç kez vurguladı. Kayıttan izlemek, sonraki yılların derslerinde (ör. algoritma analizi) eksik bırakabilir.
- Chat üzerinden soru sorulmamasını, sesli sorulmasını istedi.

### Kısa Tekrar Notları

- **Kök (root):** En üstteki düğüm. Seviyesi 0'dır.
- **Düğüm (node):** Ağaçtaki her eleman.
- **Kenar (edge):** İki düğümü bağlayan bağlantı. Bir değeri olabilir (ör. karar ağacında "evet/hayır") ya da sadece bağlantıyı gösterir.
- **İç düğüm (internal node):** En az bir çocuğu olan düğüm.
- **Yaprak (leaf):** Hiç çocuğu olmayan düğüm.
- **Derece (degree):** Bir düğümün çocuk sayısı.
- **Seviye (level):** Kök 0. seviyededir. Aynı seviyedeki düğümler hiyerarşide aynı konumdadır (ör. kardeşler).

- **Yükseklik (height):** Bir düğümün en uzak yaprağa uzaklığı. Kökün yüksekliği ağacın yüksekliğidir.
- **Derinlik (depth):** Bir düğümün köke uzaklığı.
- **Orman (forest):** Birbirinden bağımsız ağaçların oluşturduğu küme.
- **İkili ağaç:** Her düğümün en fazla iki çocuğu (sol ve sağ) olan ağaç.
- **Dizi ile ikili ağaç:** $i$. gözdeki düğümün sol çocuğu $2i + 1$, sağ çocuğu $2i + 2$ numaralı gözdedir (indis 0'dan başlar).
- **Yüksekliği $h$ olan ikili ağaç:**
  - En çok yaprak: $2^h$
  - En çok düğüm: $2^{h+1} - 1$
  - En az yaprak: 1
  - En az düğüm: $h + 1$
- **Preorder:** kök, sol, sağ (prefix ifade).
- **Inorder:** sol, kök, sağ (infix ifade).
- **Postorder:** sol, sağ, kök (postfix ifade).
- **BST kuralı:** Bir düğümün sol alt ağacındaki tüm değerler ondan küçük, sağ alt ağacındaki tüm değerler ondan büyüktür.
- **BST karmaşıklığı:** En iyi/ortalama durumda $O(\log n)$, en kötü durumda (tek yöne yığılmış ağaç) $O(n)$.

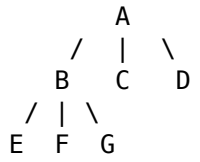
## Detaylı Açıklamalar

### Ağaç neden kullanılır?

- Bilgi hiyerarşik ise ağaç kullanılır.
- **Dosya sistemi:** Kök dizin (root directory) altında alt dizinler bulunur (ör. ödevler, oyunlar, dokümanlar). Her birinin altında yine alt dizinler olabilir.
- **Aile ağacı:** En üstte büyükanne ve büyükbaba, altında çocuklar, onların altında torunlar.
- **Karar ağacı:** Makine öğrenmesinde başarılı bir öğrenme modelidir.
  - Örnek: hastalık teşhisi. "Ateş var mı?" sorusunun cevabına göre bir dala gidilir. Sonra "Göz sarı mı?", "Öksürük ne düzeyde?" gibi sorular sorulur.
  - Hasta geldiğinde cevaplara göre ağaçta ilerlenip teşhis konur.

### Ağaç terminolojisi örneği

Hocanın tahtadaki örneği şöyleydi:




- A kök, B ve C ve D A'nın çocukları.
- A ve B iç düğümdür. A'nın derecesi 3'tür.
- C, D, E, F, G yapraktır. Yaprakların derecesi 0'dır.
- A'nın yüksekliği 2'dir. Ağacın yüksekliği de 2'dir.
- E'nin derinliği 2, B'nin derinliği 1'dir.
- Bir düğüm ve altındaki tüm düğümler bir alt ağaç (subtree) oluşturur. Tek bir yaprak da tek başına bir alt ağaçtır.

### Genel ağacın dizi ile saklanması

- Önce bir varsayım gerekir: her düğümün en fazla $k$ çocuğu olabilir (hoca örnekte $k = 3$ aldı).
- Elemanlar seviye seviye diziye yazılır. Olmayan düğümler için göz boş bırakılır.
- Boş gözler zorunludur. İleride oraya bir eleman eklenebilir ve hiyerarşinin bozulmaması gerekir.
- **Sakıncalar:**
  - En fazla çocuk sayısı önceden bilinmeli. 3 dendiyse dördüncü çocuk asla eklenemez. Dosya sisteminde kök altında 20 dizin olabilir, bunun garantisi yoktur.

- Çok sayıda boş göz oluşur, bellek israf edilir.

**Genel ağacın bağlı liste ile saklanması (left child - right sibling)**

- Her düğümde bir değer ve iki pointer bulunur:
  - **Sol çocuk (left child):** düğümün en soldaki çocuğunu gösterir.
  - **Sağ kardeş (right sibling):** aynı seviyede sağdaki kardeşi gösterir.
- Böylece bir seviyede istenen sayıda düğüm olabilir.

```
struct node {
    int value;
    struct node *leftChild;
    struct node *rightSibling;
};
```

- Örnek (A'nın çocukları B, C; B'nin çocukları D, E, F; C'nin çocuğu G):

| Düğüm | leftChild | rightSibling |
|---|---|---|
| A | B | NULL |
| B | D | C |
| C | G | NULL |
| D | NULL | E |
| E | NULL | F |
| F | NULL | NULL |
| G | NULL | NULL |

- Avantajı dinamik olmasıdır. Ör. E silinecekse D'nin sağ kardeşi F yapılır.

**İkili ağacın saklanması**

- **Dizi ile:** Elemanlar seviye seviye, soldan sağa yerleştirilir.
  - $i$. düğümün sol çocuğu $2i + 1$, sağ çocuğu $2i + 2$ numaralı gözdedir.
  - Ör. 6. gözdeki düğümün sağ çocuğu $2 \cdot 6 + 2 = 14$. gözde olur. 14. gözdeki düğümün sol çocuğu $2 \cdot 14 + 1 = 29$. gözde olur.
  - Ağaç seyrekse arada çok sayıda boş göz kalır. Hocanın örneğinde 7 elemanlı bir ağaç için en az 31 gözlük dizi gerekiyordu.
- **Bağlı liste ile:** Her düğümde değer, sol çocuk pointer'ı ve sağ çocuk pointer'ı vardır.

```
struct node {
    int value;
    struct node *left;
    struct node *right;
};
```

- Bağlı listede eleman eklemek kolaydır. Yer kullanımı da seyrek ağaçlarda daha verimlidir.

**Tam ikili ağaç (complete binary tree)**

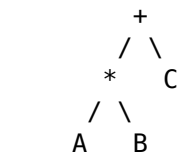
- Yüksekliği $h$ olan bir ağaç için iki şart vardır:
  1. 0'dan $h - 1$. seviyeye kadar tüm seviyeler doludur.
  2. Son seviyedeki düğümler sola dayalıdır (left justified), arada boşluk yoktur.
- Tam ikili ağaç diziye yerleştirildiğinde arada hiç boş göz kalmaz (0, 1, 2, 3, 4, 5 sırayla dolar).
- Bu yüzden tam ikili ağaç için bağlı liste gerekmez, dizi yeterlidir.

**Dengeli ikili ağaç (balanced binary tree)**

- Hocanın tanımı: Seviyeler aşağıdan yukarı $n$, $n - 1$, $n - 2$, ... diye numaralanırsa, $n - 2$. seviyeye kadar (dahil) olan tüm düğümlerin iki çocuğu olmalıdır.
- Son iki seviyedeki düğümlerin kaç çocuğu olduğuna bakılmaz.
- $n - 2$. seviyede bir düğümün çocuğu eksikse ağaç dengeli değildir (unbalanced).

### Ağaç gezinme (traversal)

Örnek ifade ağacı: $(A * B) + C$




- **Preorder (kök, sol, sağ):** `+ * A B C` (prefix)
- **Inorder (sol, kök, sağ):** `A * B + C` (infix)
- **Postorder (sol, sağ, kök):** `A B * C +` (postfix)

```
void preorder(struct node *node) {
    if (node != NULL) {
        printf("%d ", node->value);
        preorder(node->left);
        preorder(node->right);
    }
}

void postorder(struct node *node) {
    if (node != NULL) {
        postorder(node->left);
        postorder(node->right);
        printf("%d ", node->value);
    }
}

void inorder(struct node *node) {
    if (node != NULL) {
        inorder(node->left);
        printf("%d ", node->value);
        inorder(node->right);
    }
}
```

- Hoca preorder'ı ifade ağacı üzerinde adım adım izledi: `+` yazılır, sola inilir, `*` yazılır, sola inilir, `A` yazılır, NULL'lar dönülür, `B` yazılır, yukarı çıkılır, `C` yazılır.
- Inorder kodunu öğrencilere bıraktı.

### İkili arama ağacı (BST)

- Sıralı veri üzerinde ikili arama (binary search) mantığını ağaca taşır.
- Azalt ve yönet (decrease and conquer) yaklaşımıdır. Her karşılaştırmada bir alt ağaç tamamen elenir.
- Örnek: 17 kökte, 18 aranıyor.
  - 18 > 17, sağa gidilir (sol alt ağaca hiç bakılmaz).
  - 18 < 19, sola gidilir.
  - 18 bulundu. 7 elemanlı ağaçta birkaç karşılaştırma yetti.
- Karmaşıklık ağacın yüksekliği kadardır. Dengeli ağaçta $O(\log n)$.
- Hep büyük (ya da hep küçük) değerler eklenirse ağaç tek tarafa yığılır, karmaşıklık $O(n)$ olur.
- **Nerede saklanmalı?**
  - Dengesi ayarlanmış, üzerinde az değişiklik olan bir ağaç dizide saklanabilir.
  - Sürekli ekleme/silme yapılıyorsa bağlı liste kullanılmalıdır.

### BST'ye ekleme (insert), döngü ile

- Yeni eleman için düğüm oluşturulur (`tmp`), sol ve sağ pointer'ları NULL yapılır.
- Ağaç boşsa (`root == NULL`) kök bu düğüm olur.

- Değilse kökten başlanır. Değer küçükse sola, büyükse sağa gidilir. Gidilen yerin ebeveyni (`parent`) saklanır.
- Gidilen yer NULL ise yeni düğüm ebeveynin soluna veya sağına bağlanır.

```
struct node *insert(struct node *root, int x) {
    struct node *tmp = malloc(sizeof(struct node));
    tmp->value = x;
    tmp->left = tmp->right = NULL;

    if (root == NULL)
        return tmp;              /* ağaç boştu, yeni kök */

    struct node *current = root, *parent;
    int stop = 0;
    while (!stop) {
        parent = current;
        if (x < current->value) {
            current = current->left;
            if (current == NULL) { parent->left = tmp; stop = 1; }
        } else {
            current = current->right;
            if (current == NULL) { parent->right = tmp; stop = 1; }
        }
    }
    return root;
}
```

- Hocanın adres örneği (9 kökte, 1000 numaralı adreste; 5 → 1500, 11 → 1700, 17 → 1900; yeni düğüm 800):
  - **3 eklenirken:** 3 < 9, parent = 1000, current = 1500 (5). 3 < 5, parent = 1500, current = NULL. 5'in soluna 800 (3) bağlanır.
  - **21 eklenirken:** 21 > 9, current = 1700 (11). 21 > 11, current = 1900 (17). 21 > 17, current = NULL. 17'nin sağına 800 (21) bağlanır.

### BST'de arama (search)

```
struct node *search(struct node *root, int value) {
    struct node *current = root;
    while (current != NULL && current->value != value) {
        if (value < current->value)
            current = current->left;
        else
            current = current->right;
    }
    return current;   /* NULL ise bulunamadı */
}
```

- Fonksiyon bulunan düğümün adresini döndürür. NULL dönerse çağıran taraf "bulunamadı" der.
- Örnek: 13 aranırken 17'den sola (11), 11'den sağa gidilir ve 13 bulunur. 9 aranırken 17 → 11 → 6 → 6'nın sağı NULL, bulunamadı.

### Çift pointer sorusu

- Bir öğrenci bazen çift pointer, bazen tek pointer kullanıldığını sordu.
- Hoca bu derste hiç çift pointer kullanmadığını söyledi.
- Çift pointer, pointer'ın pointer'ıdır. Bir pointer değişkenin adresini tutar.
- Örnek: `a = 5` (adres 1250), `y = &a` (y'nin değeri 1250, kendi adresi 1800), `x = &y` (x'in değeri 1800). Bu durumda `b = **x;` ifadesi önce 1800'e, oradan 1250'ye gider ve `b` 5 olur.

**Ders sonrası notlar**

- **Postorder sonucu:** Hoca postorder'ı anlatmaya başlarken $(A * B) + C$ için önce "artı çarpı A B C" dedi. Bu preorder sonucudur. Doğru postorder sonucu `A B * C +`'dır. Hoca adım adım izlemede doğru sonucu buldu.
- **Çift pointer örneği:** Hoca örnekte "x = y" dedi. `x`'in `y`'nin adresini (1800) tutması için doğru yazım `x = &y;` olmalıdır. `b = **x;` ifadesi ancak `x` bir `int **` ise anlamlıdır.
- **Çift pointer gerçekte nerede gerekir:** Bir fonksiyonun kök pointer'ını değiştirmesi gerekiyorsa (ör. boş ağaca ilk eleman eklenirken) ya yeni kök geri döndürülür ya da `struct node **root` kullanılır. Yukarıdaki `insert` kodu yeni kökü döndürerek bu sorunu çözer. Öğrencinin daha önce gördüğü çift pointer kullanımları bu tür durumlardan gelir.
- **Tam ikili ağaç ile dengeli ağaç:** Hocanın "dengeli ağaç" tanımı (son iki seviye hariç tüm düğümlerin iki çocuğu olması) bazı kaynaklarda farklı tanımlanır. Ör. AVL ağacında denge, her düğümde sol ve sağ alt ağaç yükseklik farkının en çok 1 olmasıdır. Sınavda hocanın tanımı esas alınmalıdır.

**Duyurular**

- Bu ders üniversiteden yapıldı. Sınıfta kimse olmadığı için hoca odasından online anlattı. Ders hibrit (yüz yüze + online) yürütülmeye çalışılacak.
- Sınavların online olacağı söylendi.
- Uygulama/laboratuvar dersleri hakkında henüz bölümden bilgi gelmediği söylendi. Ertesi günkü uygulama yapılmayacak. Sonraki haftalarda online soru çözümü şeklinde yapılabilir.

**Not:** İsterseniz bu dersin altyazı (.srt) dosyasını NotebookLM gibi bir yapay zeka aracına yükleyerek ders hakkında daha detaylı soru-cevaplar yapabilir ve dersi verimli çalışabilirsiniz.